SahiReport — Security & Launch Readiness Report

Pre-launch review of the Flutter (Client) + Supabase (Backend) field-reporting application

Repository: **SahiReport** · Version reviewed: 1.0.0+1 · Review date: 28 October 2026 · Method: static review of `lib/`, platform config, Edge Functions, and git history

> **Verdict: NOT ready for public launch.**
>
> Five controls that determine whether revenue and customer data are protected are missing or unverifiable. Two of them independently block Google Play / App Store submission. Closed beta is defensible only after C1, C3, C4 and C5 are closed. Do not switch Razorpay to a live key before C1 is fixed.

6	7	5	18
CRITICAL	HIGH	MEDIUM	TOTAL FINDINGS

ID	FINDING	SEVERITY	EVIDENCE
C1	**Pro entitlement granted by the client; no payment verification** The Razorpay callback only proves the SDK reported success. The app then writes `plan_type`, `pro_project_slots` and `pro_expires_at` itself. A modified client can grant itself unlimited Pro slots for free.	CRITICAL	`billing_screen.dart:46-89`
C2	**Release builds signed with the debug keystore** The debug key is public and identical across all Android SDK installs. Play rejection risk; third parties can sign a fake "update" that Android treats as the same app.	CRITICAL	`android/app/build.gradle.kts:31-35`
C3	**RLS policies absent from the repository; security boundary unverifiable** All non-owner access reaches PostgREST as `anon` using a key embedded in every APK. If RLS is missing or permissive, all customer data, GPS records, photo URLs and FCM tokens are exposed.	CRITICAL	no `supabase/migrations/`; schema exists only in hosted project
C4	**Public storage bucket with unauthenticated uploads** Field users upload without a session and the app calls `getPublicUrl()`. Anyone holding the anon key can upload arbitrary files (cost abuse / illegal content), and every photo URL is permanently public — including after account deletion.	CRITICAL	`submission_repository.dart:168-174, 224-230`

ID	FINDING	SEVERITY	EVIDENCE
C5	**Guessable co-owner code; no expiry, no throttling** Co-owner codes are `ADMN-` + 4 characters from a 32-symbol alphabet (≈1.05M combinations). The login RPC is callable by `anon` with no attempt limit; a successful code grants the whole owner account and never expires.	CRITICAL	`co_owner_repository.dart:9-15; auth_provider.dart:322-365`
C6	**"GPS-verified / tamper-proof" is asserted entirely by the client** Coordinates, timestamp and the photo stamp are produced on-device. A modified client can submit any location and any image. The commercial claim is not backed by a server-side trust boundary.	HIGH — PRODUCT/TRUST	`image_stamp_service.dart; submit_report params`
H1	**"Delete account" only soft-deletes; hard-delete function never called** `is_deleted` / `deleted_at` are set and the user is signed out. The `delete-account` Edge Function is never invoked, so no data is actually removed — contradicting the published policy.	HIGH — LEGAL	`auth_repository.dart:165-170; profile_edit_screen.dart:151`
H2	**Privacy policy is an unfilled template and contradicts the app** Placeholders remain (`[DATE]` , `[COMPANY LEGAL NAME]` , `[PRIVACY CONTACT EMAIL]` , `[REGION]`). It states data is not used for third-party advertising while AdMob banners are displayed. No consent step at signup.	HIGH — LEGAL	`Privacy Policy.txt:27-44, 89-90, 156-158`
H3	**Push-notification function callable by anyone; no ownership check** Uses the service role and trusts the request body. Any holder of the anon key can trigger notification spam against owners and burn FCM quota.	HIGH	`functions/send-notification/index.ts:15-21`
H4	**Digital goods sold via Razorpay inside the app** Google Play requires Play Billing for in-app digital goods; Apple requires IAP. Current flow is a removal risk. Either integrate store billing or sell externally and only consume entitlement in-app.	HIGH — STORE POLICY	`billing_screen.dart:103-132`
H5	**Background location declared but never used** The app only calls `getCurrentPosition` (when-in-use). Play requires a background-location declaration and video; Apple rejects unused "Always" requests.	HIGH — STORE REVIEW	`AndroidManifest.xml:9; ios/Runner/Info.plist:35`

ID	FINDING	SEVERITY	EVIDENCE
H6	**Owner session stored unencrypted; device backups enabled** `flutter_secure_storage` is declared but never used, so supabase_flutter persists the session in SharedPreferences. Android `allowBackup` is not disabled.	HIGH	`pubspec.yaml:63`; no `FlutterSecureStorage` usage; manifest has no `allowBackup`
H7	**Free-tier submission cap is not enforced** `freePlanMaxSubmissions = 50` is referenced nowhere. There is no submission throttling, leaving storage-cost and spam abuse unmitigated.	HIGH — COST ABUSE	`app_constants.dart:44` — unused
M1	**Profile update writes NULL into every omitted column** The update map includes all fields unconditionally, so partial updates erase existing data.	MEDIUM	`auth_repository.dart:146-157`
M2	**Image upload failures are swallowed** Per-image errors are only logged, so a "successful" report can contain zero evidence photos.	MEDIUM	`submission_repository.dart:184-186, 239-241`
M3	**Co-owner can reach the billing route** Billing is hidden in the UI but not blocked in the router redirect; a co-owner could purchase on the owner's account.	MEDIUM	`app_router.dart:45-98`; `owner_dashboard_screen.dart:1197-1221`
M4	**Account deletion never removes Storage objects** The hard-delete function removes table rows only; photos remain in the bucket.	MEDIUM	`functions/delete-account/index.ts:65-91`
M5	**No server-side OTP send throttling** Only a 30-second client cooldown exists; it is bypassable and does not protect SMS/email spend.	MEDIUM	`confirm_email_screen.dart:26-62`
M6	**No iOS privacy manifest** Required-reason API declarations are missing, which is a practical App Store rejection risk.	MEDIUM	no `PrivacyInfo.xcprivacy` in the repository

Verified clean

- No service-role key or private key in git history. The `-S` matches were code strings inside the Edge Functions.

- The Supabase key in `app_constants.dart` decodes to `role: anon` — public by design, not a leaked secret. RLS is therefore the only security boundary.
- Auth error messages for owner login are generic (no account enumeration).
- The `delete-account` function correctly verifies the caller's JWT before acting.

Required before launch — in order

1. **Server-side payment verification + column-level grants** (C1). Add a `verify-payment` Edge Function (HMAC-SHA256 of `order_id|payment_id`) or a Razorpay webhook; revoke `UPDATE` on billing columns from `authenticated`.
2. **Real upload keystore** (C2): `key.properties` + `signingConfigs.release`; never commit it.
3. **Version the schema and audit policies** (C3): `supabase db pull` into `supabase/migrations/`, then run the checks in the appendix.
4. **Private bucket + signed URLs** (C4), issued only after server-side code validation.
5. **8+ character expiring codes**, hashed at rest, with per-IP/per-code attempt lockout (C5).
6. **Resolve store policy** (H4), remove unused permissions (H5), complete the privacy policy and Data Safety form (H2), add `PrivacyInfo.xcprivacy` (M6).
7. **Make deletion real** (H1, M4) or disclose soft-delete and add a purge job.
8. **Harden the notification function and rate limits** (H3, H7, M5); fix or qualify the "tamper-proof" claim (C6).

Static review only. Database policies, RPC definitions, triggers, bucket flags and Razorpay webhook configuration were not available in the repository and are listed as unverified in C3/C4. No live endpoint was queried for this report. Prepared for the SahiReport maintainers.

Appendix — Verification SQL (run in Supabase SQL Editor)

Run against a staging project first. Expected safe results: every table with `relrowsecurity = true`; zero policies granting `anon`; no `UPDATE` on billing columns; `login_*` functions `prosecdef = true` and filtering `is_active`; storage bucket `public = false`.

```
-- 1. Is RLS enabled on every table?
select relname, relrowsecurity
from pg_class c
join pg_namespace n on n.oid = c.relnamespace
where n.nspname = 'public' and c.relkind = 'r'
order by 1;

-- 2. Every policy that grants the anon role
select tablename, policyname, cmd, roles::text, qual, with_check
from pg_policies
where schemaname = 'public' and 'anon' = any(roles)
order by 1, 2;

-- 3. Can an owner rewrite their own billing columns?
select grantee, privilege_type, column_name
from information_schema.column_privileges
where table_name = 'owners'
  and grantee in ('anon', 'authenticated')
  and privilege_type = 'UPDATE';

-- 4. Are the RPCs SECURITY DEFINER, and do they check is_active?
select p.proname, p.prosecdef, pg_get_functiondef(p.oid)
from pg_proc p
join pg_namespace n on n.oid = p.pronamespace
where n.nspname = 'public'
  and p.proname in ('login_with_code', 'login_co_owner_with_code',
                    'submit_report', 'add_submission_image',
                    'get_user_submissions', 'generate_access_code');

-- 5. Is the bucket public, and who can write?
select id, name, public from storage.buckets;
select policyname, cmd, roles::text, qual, with_check
from pg_policies
where schemaname = 'storage' and tablename = 'objects';
```

Golden test (staging only)

Sign in as an ordinary owner, then from the debug console run:

```
update owners
set plan_type = 'pro', pro_project_slots = 999
where id = auth.uid();
```

If this succeeds, C1 is confirmed exploitable in production. It must fail with a permission error once column-level grants are in place.

Severity legend

- **CRITICAL** — exploitable now; enables data breach, free entitlements or unrecoverable key compromise.
- **HIGH** — likely exploitation or a hard store/legal rejection; fix before launch.
- **MEDIUM** — real impact but needs a specific condition or an insider; fix in the first release window.